

2. Echauffement

Combien de connexions TCP sont contenues dans la trace ?

Utilisez le menu **statistics -> conversations** pour répondre à cette question.

Une seule

Isolez la connexion débutant à la trame 1.

Clique droit dessus → Follow → TCP Stream.

Quel est son contenu ?

Le contenu d'une connexion TCP (**après le 3-way handshake**) dépend de ce qui est transporté :

- Si c'est du **HTTP**, tu verras des requêtes GET/POST et des réponses HTML.
- Si c'est du **HTTPS**, le contenu sera chiffré (tu verras juste du TLS).
- Si c'est une autre appli, tu verras son protocole (FTP, SSH, etc.).

Dans notre cas c'est du **HTML**

Quels sont les numéros des trames débutant cette connexion (3 way handshake) ?

1, 2 et 3

Toujours pour la même connexion TCP, affichez les numéros de séquence et numéros d'ACK en tant que colonne.

Pour cela, dans la fenêtre détaillant le contenu de la trame sélectionnée, cliquez sur le champ souhaité et choisissez "Apply as column".

Ajouter les colonnes pour bien voir les numéros

1. Clique sur une trame de ta connexion (par ex. la trame 1).
2. Dans le panneau du bas ("Frame details"), développe **Transmission Control Protocol**.
3. Clique droit sur :
 - **Sequence number (raw)** → Apply as column
 - **Acknowledgment number (raw)** → Apply as column
4. Ces champs apparaissent comme nouvelles colonnes dans ta vue principale.

Quels sont les numéros de séquence utilisés dans la direction client => serveur ?

Combien de données par segment ?

A quoi correspondent ces données ?

[illegible]

Dans le sens **client** → **serveur**, les segments contiennent **0 octet de données** : ce sont uniquement des **segments de contrôle TCP (SYN, ACK, etc.)**.

interprétation

Le **premier segment** : incrément de 1 → probablement un petit segment ou un ACK.

Deuxième segment : incrément 725 → petite charge utile (payload de 725 octets).

Segments suivants : incrément = 0 → ce sont des **répétitions / retransmissions / segments sans nouvelle donnée**.

Dernier segment : incrément = 1 → fin de flux ou petit segment final.

Même question dans le sens serveur => client

Numéros de séquence	Données par segment
42 00 11 12 40	1 bytes
42 00 11 12 41	1448 bytes
42 00 11 26 89	1448 bytes
42 00 11 41 37	1448 bytes
42 00 11 55 85	1448 bytes
42 00 11 70 33	1448 bytes
42 00 11 84 81	1448 bytes
42 00 11 99 29	1448 bytes
42 00 12 13 77	1448 bytes
42 00 12 28 25	1448 bytes
42 00 12 42 73	1448 bytes
42 00 12 57 21	1448 bytes
42 00 12 71 69	1448 bytes
42 00 12 86 17	1448 bytes
42 00 13 00 65	1448 bytes
42 00 13 15 13	1448 bytes
42 00 13 29 61	1230 bytes
42 00 13 41 91	Fin

interprétation

Le **premier segment 42 00 11 12 40 – 0 octet** → segment de contrôle (SYN-ACK ou simple ACK).

Deuxième segment 42 00 11 12 41 – 1 octet → très petit segment (souvent une réponse initiale, ex. un accusé de protocole applicatif).

Puis une série de segments de 1448 octets →

- Cela correspond à la **taille maximale de données TCP (MSS)** permise par le chemin réseau.
- Ici, le serveur envoie des blocs de 1448 octets chacun, une **réponse HTTP**

Dernier segment = 1230 octets → la fin de la réponse ne remplissait pas un segment complet (reste de la donnée à envoyer).

En résumé : côté serveur, on observe un gros flux de données continues, envoyées par blocs maximum de 1448 octets, jusqu'au dernier segment plus petit. Cela correspond très typiquement à un **téléchargement de contenu (page web, ressource, fichier, etc.)**.

Quand les segments ACK sont-ils envoyés ?

Un segment qui ne contient que l'accusé de réception (pas de données).

Envoyé quand une machine doit confirmer la bonne réception de données, mais qu'elle n'a pas de données à envoyer en retour immédiatement.

Discutez les différents cas de figure.

1. ACK pur (standalone ACK)

Un segment qui ne contient que l'accusé de réception (pas de données).

Envoyé quand une machine doit confirmer la bonne réception de données, mais qu'elle n'a pas de données à envoyer en retour immédiatement.

2. ACK piggyback (ACK combiné avec des données)

Pour éviter d'envoyer trop de paquets « vides », TCP essaie de combiner l'ACK avec des données sortantes.

Exemple : si le client reçoit des données et doit en renvoyer, il place l'ACK dans l'entête de son propre segment de données.

Cela économise de la bande passante et réduit le trafic inutile.

3. ACK retardé (delayed ACK)

Souvent, les systèmes n'envoient pas l'ACK immédiatement après chaque segment reçu.

Ils attendent un court délai (typiquement 40–200 ms selon les implémentations) pour voir si :

- ils peuvent envoyer des données en retour (ACK piggyback),
- ou s'ils reçoivent un second segment (dans ce cas, un seul ACK couvrira les deux).

Exemple : si deux paquets arrivent rapidement l'un après l'autre, un seul ACK suffit.

⚠ **Mais attention** : si aucun segment ou donnée n'arrive pendant ce délai, un ACK « pur » est envoyé avant que le timer expire.

4. ACK cumulatifs

Le numéro d'acquittement (Acknowledgment Number) dans TCP est **cumulatif** : il confirme la réception de **toutes les données jusqu'à un certain octet**.

Exemple : si j'ai reçu jusqu'à l'octet 3000 inclus, j'envoie ACK = 3001, même si plusieurs segments distincts m'ont amené jusque-là.

Cela évite d'envoyer un ACK pour chaque segment.

5. ACK rapides (fast ACK) dans le cas de pertes

Lorsqu'un segment est perdu, le destinataire continue de recevoir les segments suivants, mais il envoie **plusieurs ACK du même numéro** (duplication d'ACK).

Exemple : il a reçu jusqu'à l'octet 2000, perd le segment [2001–3000], mais reçoit [3001–4000]. Il renvoie plusieurs fois ACK = 2001 pour signaler le trou.

L'émetteur, voyant plusieurs ACK dupliqués (3 ACK identiques), comprend qu'un segment est perdu et retransmet immédiatement (mécanisme **fast retransmit**).

6. ACK pendant l'établissement et la terminaison de connexion

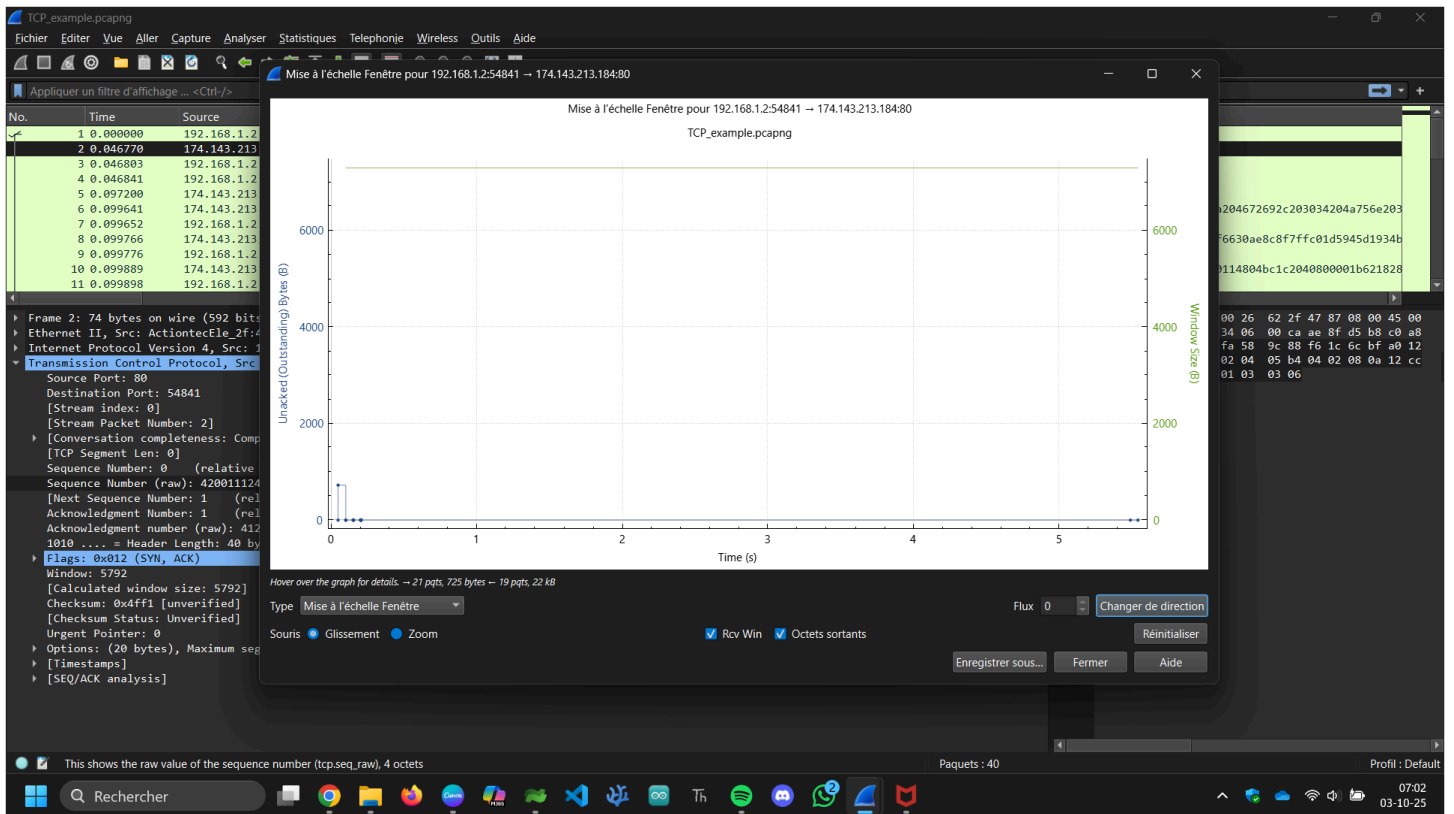
3-way handshake : le 3^e segment est un ACK qui confirme le SYN+ACK.

Fermeture de connexion : chaque FIN doit être accusé réception par un ACK.

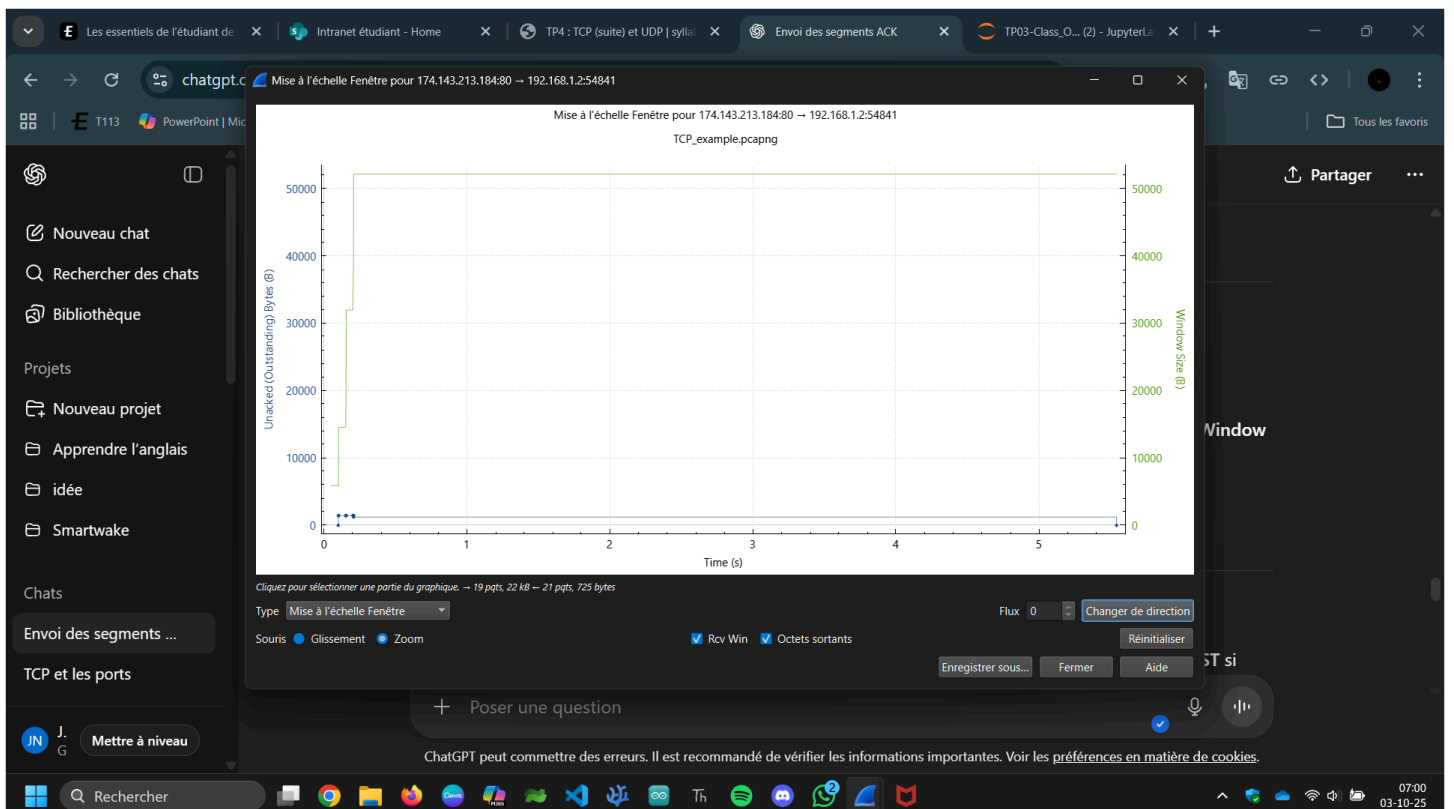
Toujours pour la même connexion TCP, comment évolue la taille de fenêtre, dans le client vers serveur ?

Utilisez pour cela le menu Statistics => TCP stream graphs => Window Scaling.

Taille de fenêtre, client vers serveur	Taille de fenêtre, serveur vers client
5840	5792
46	114
46	114
69	114
91	114
114	114
137	114
159	114
182	114
204	114
227	114
250	114
272	114
295	114
318	114
340	114
363	114
385	114
408	114
408	114



Et dans le sens serveur vers client ?



Qu'est ce qui déclenche ces variations ?

Évolution de la taille de fenêtre (client → serveur)

Elle démarre à **5840** (valeur initiale négociée à l'ouverture de la connexion).

Puis elle chute très fortement à **46** et reste faible un moment.

Ensuite, elle **augmente progressivement** : 69, 91, 114, 137, 159... jusqu'à environ **408**, où elle se stabilise.

Cela traduit un **ajustement progressif du buffer de réception côté client**, en fonction des données reçues et de la vitesse à laquelle elles sont traitées.

Évolution de la taille de fenêtre (serveur → client)

Elle commence à **5792** (valeur initiale).

Ensuite, elle tombe à **114** et reste **quasi fixe** (114 tout du long).

Cela signifie que **le serveur limite très rapidement sa fenêtre de réception** et la garde à une taille réduite, probablement parce que son application consomme les données lentement, ou parce que le buffer reste presque rempli.

Qu'est-ce qui déclenche ces variations ?

La **taille de la fenêtre TCP** (champ *Window*) représente l'espace libre restant dans le **buffer de réception** de chaque machine.

Chaque fois que l'application lit ou non les données du buffer, la taille annoncée varie :

- **Augmentation** de la fenêtre → le destinataire a lu des données, donc de la place se libère.
- **Diminution** de la fenêtre → le buffer se remplit, le destinataire n'a pas encore tout traité.

Ici :

Côté **client**, la fenêtre évolue dynamiquement → il lit progressivement les données.

Côté **serveur**, la fenêtre tombe vite à 114 et reste fixe → cela reflète une **limitation du débit** côté serveur (soit volontaire, soit lié à une lenteur de lecture des données par son application).

Comment se termine cette connexion TCP ?

Dans ta capture, la connexion TCP **se termine en 3 segments** (et pas 4).

Ce cas correspond à :

- **Segment 1** : un côté (par ex. le client) envoie **FIN, ACK** → il annonce qu'il a fini d'envoyer des données.

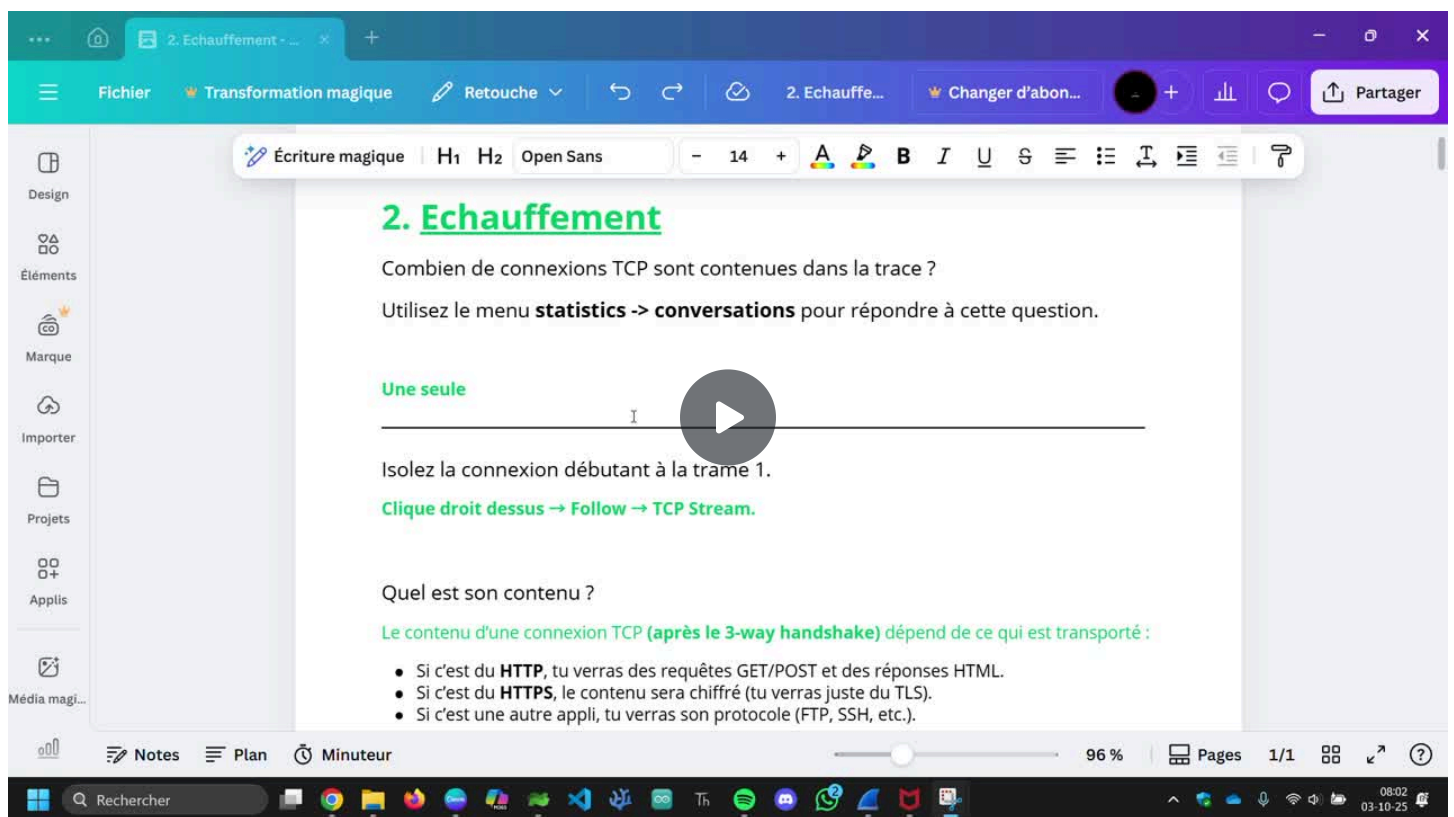
- **Segment 2** : l'autre côté (par ex. le serveur) répond **FIN, ACK** dans le même segment → il accuse réception *et* annonce aussi qu'il a fini.
- **Segment 3** : le premier côté envoie un simple **ACK** → il confirme la fermeture.

👉 Donc au lieu d'avoir deux étapes séparées (FIN puis plus tard un autre FIN), les deux se combinent dans un seul segment : c'est une **fermeture simultanée**.

Donnez le numéro de la/les trames impliquée(s).

38, 39 & 40

Remarque : Vous pouvez visualiser l'échange de manière graphique grâce au menu Statistics => Flow Graph (sélectionnez les options ad-hoc).



3. Retransmissions

Ouvrez la trace [tcp_retransmission.pcapng](#) (src : Chris Sanders).

Cette trace montre une retransmission TCP. Sélectionnez une des trames surlignée en noir. Remarquez que Wireshark vous propose une section "SEQ/ACK analysis", dans laquelle il est mentionné que la trame est une possible retransmission.

Qu'est-ce que c'est la section SEQ/ACK analysis

- Cette section est un **outil d'analyse interne de Wireshark pour TCP**.
- Elle **explique comment le segment s'insère dans la séquence TCP**, et si ce segment est :
 - une **retransmission**
 - une **perte de segment**
 - ou un **segment hors ordre**
- Elle aide à **déboguer TCP** sans avoir besoin de calculer manuellement les numéros de séquence et d'ACK.

Ce qui est contenu dans cette section

Pour chaque segment TCP sélectionné, Wireshark peut afficher :

1. **[SEQ/ACK]**
 - Montre le **numéro de séquence** du segment et le **numéro ACK attendu**.
 - Permet de vérifier si le segment est dans le bon ordre ou non.
2. **Relative sequence number / Relative ACK number**
 - Les numéros sont souvent affichés **relatifs** au premier segment de la connexion (pour simplifier la lecture).
3. **TCP segment length**
 - La taille des données utiles (payload) dans le segment.
4. **TCP window size**
 - La taille de la fenêtre annoncée par le destinataire pour ce segment.
5. **Flags liés aux retransmissions**
 - **[TCP Retransmission]** → segment réexpédié (même SEQ qu'un segment précédent non acquitté)
 - **[TCP Fast Retransmission]** → retransmission rapide détectée par triple ACK
 - **[Out-of-Order]** → segment arrivé hors séquence
 - **[Previous segment not captured]** → Wireshark n'a pas vu le segment précédent, attention possible erreur
6. **RTO / Time since last segment**
 - Le **temps écoulé depuis l'envoi du segment précédent**, utile pour comprendre si la retransmission est due au timeout.

D'après vous, sur quoi se base Wireshark pour cela ?

Wireshark met en évidence une **possible retransmission** en comparant :

- Le **numéro de séquence (SEQ)** d'un segment TCP
- Avec les **numéros de séquence précédemment vus dans la même connexion**

Si Wireshark voit un segment avec le **même SEQ** qu'un segment déjà envoyé **et non encore acquitté (ACK)**, il le marque comme retransmission.

En clair :

- **Même numéro de séquence + pas de nouvel ACK → retransmission.**
- **Il ne s'agit pas d'une confirmation absolue, mais d'une probabilité très élevée.**

Que signifie le RTO indiqué ?

RTO = Retransmission Timeout (temps avant retransmission).

- C'est le **délai** que **TCP attend après l'envoi d'un segment** avant de le renvoyer si aucun ACK n'est reçu.
- Il est calculé dynamiquement par TCP, basé sur le **RTT (Round-Trip Time)** moyen et sa variance.

Comment évolue-t'il au fur et à mesure des retransmissions ?

TCP utilise une **stratégie exponentielle** : à chaque retransmission, le RTO **double** (back-off exponentiel).

Exemple :

- 1^{re} retransmission → RTO initial (ex : 200 ms)
- 2^e retransmission → RTO = 400 ms
- 3^e retransmission → RTO = 800 ms
- ... et ainsi de suite

⚡ Cela permet d'éviter d'inonder le réseau si le segment ne passe pas.

Quel type de problème semble indiquer cette trace ?

The image shows a Wireshark packet capture of a TCP connection. The packet list at the top shows a segment at 0.000000 and a retransmission at 0.000000. The packet details pane shows the segment's metadata, including sequence number 1, acknowledgment number 1, and flags PSH, ACK. The packet bytes pane shows the segment's data. A red box highlights the 'Retransmitted TCP segment data (648 bytes)' section. A red arrow points from the 'Retransmitted TCP segment data' section to the 'Retransmission' section in the packet list.

Ce que signifient les lignes que l'on voit

Ligne que tu vois	Signification
bytes in flight: 648	Le segment contient 648 octets de données utiles .
byte sent since last PSH flag: 648	Ces 648 octets ont été envoyés avec le flag PSH (Push), indiquant que l'application voulait que les données soient transmises immédiatement.
TCP Analysis Flags	C'est la section où Wireshark analyse le comportement TCP pour détecter des anomalies (retransmissions, hors ordre, etc.).
Expert info (Notes/Sequence): This frame is a (suspected) retransmission	Wireshark suspicion de retransmission : le segment a le même numéro de séquence qu'un segment précédent et l'ACK correspondant n'a pas été reçu.
[Severity level: Note] [Group Sequence]	Niveau de sévérité de l'alerte (ici, c'est juste un warning, pas une erreur critique).
[RTO based on delta from frame: 1]	Cette retransmission est déclenchée par un timeout RTO calculé à partir du premier segment envoyé (Round Trip Time estimé).

Que cela nous dit sur le problème réseau

Retransmission détectée → segment envoyé **une première fois**, mais **aucun ACK reçu**. TCP doit renvoyer le segment.

RTO → la retransmission est **due à un timeout**, pas à un triple ACK (qui déclencherait une **fast retransmission**).

Même numéro de séquence → le segment renvoyé est exactement le même que le précédent.

Donc, le type de problème que cela indique :

Perte de paquets sur le réseau

- Le segment original n'a jamais atteint le destinataire, ou
- L'ACK du segment original a été perdu.

Lenteur ou congestion réseau

- Le RTO augmente à chaque retransmission (back-off exponentiel).

- Cela montre que TCP doit attendre de plus en plus longtemps avant de renvoyer le segment.

Pas de corruption détectée

- Wireshark ne signale pas d'erreur CRC ou checksum → la perte est probablement **due au réseau**, pas aux données elles-mêmes.

Ouvrez à présent la trace [tcp_dupack.pcapng](#)(source : Chris Sanders). Observez également les informations données dans la section "SEQ/ACK analysis".

Combien y a-t-il de segments identiques ?

4

Pourquoi y a-t-il duplication d'ACK ?

La duplication d'ACK apparaît quand le récepteur reçoit un ou plusieurs segments **hors-séquence** parce qu'un segment intermédiaire a été perdu (ou fortement retardé).

Le récepteur ne peut pas avancer son pointeur d'octet attendu, donc il renvoie à chaque paquet hors-séquence un ACK répétant le **numéro d'acknowledgement du dernier octet reçu en ordre**. En pratique : si le segment N est perdu et que N+1, N+2 arrivent, le récepteur renvoie plusieurs fois l'ACK pour le dernier octet correct (celui avant N) — d'où les *duplicate ACKs*.

Le problème finit-il par se résoudre ?

Oui — dans cette trace typique le problème se résout : après réception de **trois (ou plus) duplicate ACKs**, l'émetteur déclenche une **retransmission rapide** du segment manquant (plutôt que d'attendre le délai de retransmission RTO). Une fois le segment retransmis et reçu, le récepteur peut avancer son ACK, les ACKs dupliqués cessent et la connexion reprend. On observe souvent aussi une réduction de la fenêtre de congestion (cwnd) — la connexion ralentit momentanément, puis reprend.

Comment s'appelle le mécanisme montré dans cette trace ?

Le mécanisme s'appelle **Fast Retransmit** (retransmission rapide).

Il indique que le récepteur attend toujours le même octet (le segment manquant) et renvoie donc plusieurs fois le même numéro d'ACK.

Quand on voit **trois duplicate ACKs** consécutifs, c'est le signal classique qui provoque chez l'émetteur une **Fast Retransmit** (retransmission rapide) du segment manquant au lieu d'attendre le RTO.

Quelle est la différence entre ce scénario et le précédent ?

👉 Premier scénario

C'était un échange TCP tout à fait **normal**, avec l'établissement de connexion (**SYN → SYN-ACK → ACK**) et ensuite des segments de données échangés sans aucune perte.

Le client répond avec des **ACKs normaux** après réception des données du serveur.

Résultat : aucun paquet dupliqué, aucune retransmission, aucun message d'erreur. C'est la séquence TCP idéale, fluide et propre.

👉 Deuxième scénario

Ici, on a bien une **perte de segment** : le récepteur n'avance plus son ACK et renvoie **des ACK dupliqués** (duplicate 1, 2, 3).

Après trois dupACKs, l'émetteur détecte la perte et déclenche une **Fast Retransmit** du segment perdu.

Une fois ce segment reçu, le récepteur peut enfin avancer son ACK et le flux reprend normalement.

✅ Donc la **grande différence** entre les deux scénarios, c'est :

Scénario 1 : fonctionnement parfait, pas de pertes ni duplications.

Scénario 2 : perte réelle → dupACKs → fast retransmit pour corriger.

Essayez, sur base d'une trace que vous avez vous-même capturée, de retrouver des retransmissions de segments, des segments perdus, ou des segments dupliqués.

Quels filtres pouvez-vous utiliser pour cela ? (indice : tcp.analysis.*).

Filtres de base (copier / coller dans la barre de filtre)

Retransmissions (toutes)

tcp.analysis.retransmission

Fast retransmit (déclenchée par 3 dupACKs)

tcp.analysis.fast_retransmission

Retransmission suspectée / possible (parfois utilisée par Wireshark)

tcp.analysis.spurious_retransmission

ACK dupliqués (dupACKs)

tcp.analysis.duplicate_ack

Segments arrivés hors ordre (out-of-order)

tcp.analysis.out_of_order

Segment perdu (déduit par Wireshark)

tcp.analysis.lost_segment

Réassemblage / segment dupliqué (duplicate segment — données dupliquées)

tcp.analysis.duplicate_ack || tcp.analysis.retransmission

Essayez d'expliquer vos observations.

-- Video D'explication Pas encore fait : j'ai la flemme --

4. Les options TCP

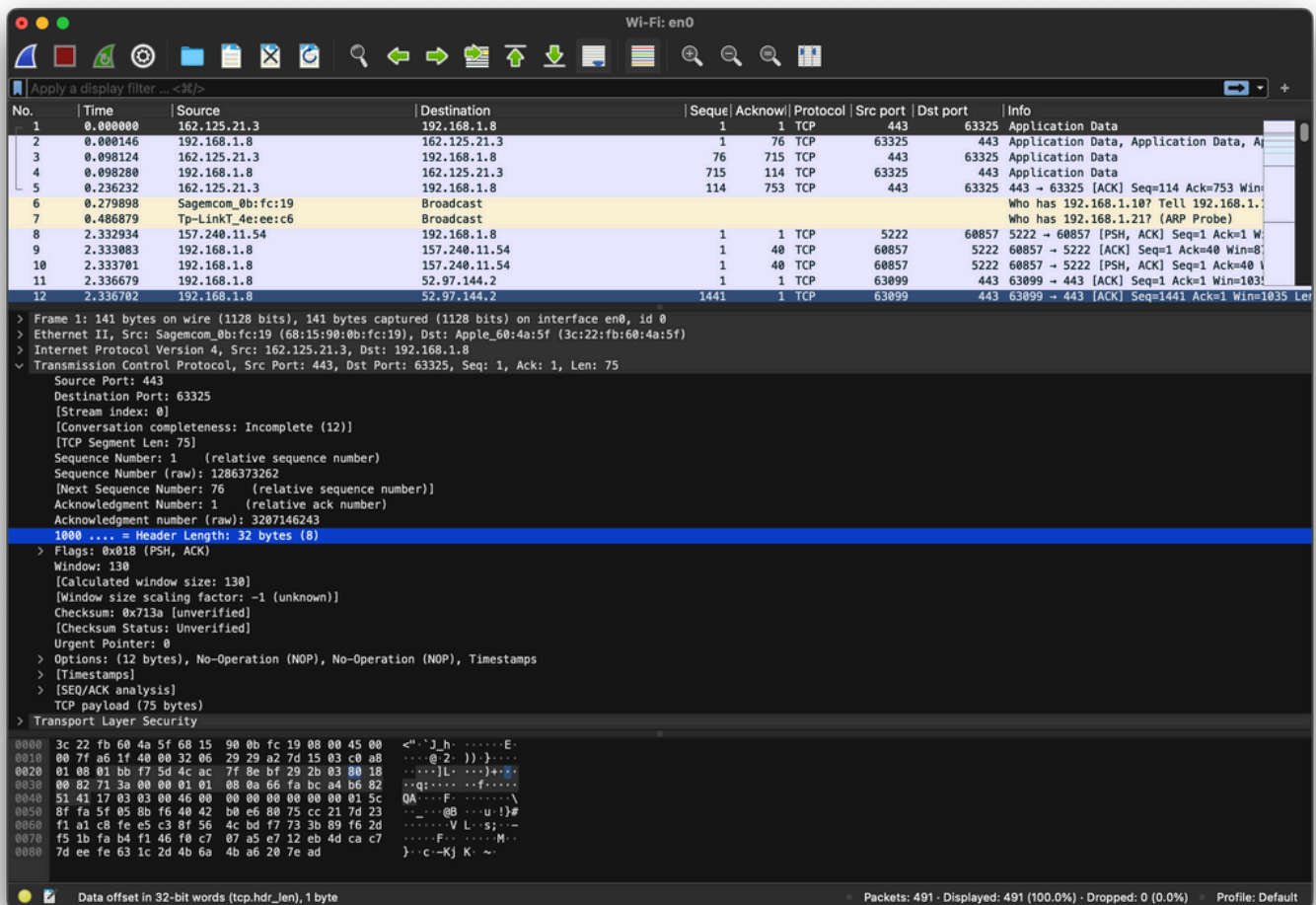
Introduction

Nous allons à présent examiner de plus près les options TCP, présents de manière facultative à la fin de l'en-tête d'un segment.

Pour déterminer si l'en-tête TCP contient des options, il faut examiner le champ "Data Offset".

Ce dernier permet d'identifier où se situe le début des données, et donc, par extension, identifie la longueur de l'en-tête.

Si sa valeur est supérieure à 20 octets (somme des tailles des champs obligatoires), alors l'en-tête contient un ou plusieurs champs d'options.



Dans l'image ci-dessus, le champ "Data Offset" (dénommé "**Header Length**" par Wireshark) vaut 32 octets.

On en déduit donc qu'il y a des options TCP, et qu'elles occupent 12 octets.

Questions

Pour chaque option listée ci-dessous, essayez de trouver dans une trace :

1. des segments portant cette option
2. la longueur de l'en-tête TCP avec cette option
3. le type de segment contenant cette option (ex : ouverture de connexion, ou bien échange de données)
4. Quelques exemples de valeurs pour les champs de l'option
5. Une explication de l'utilisation de l'option et la signification de ses différents champs

Options à investiguer : MSS, SACK, Timestamp, Window Scale, No Operation

1. Option MSS (Maximum Segment Size)

Nom du filtre Wireshark :

tcp.options.mss_val

Segments concernés :

→ Présente **dans les segments SYN** (phase d'ouverture de connexion).

Longueur d'en-tête TCP :

En général **32 octets** (20 de base + 12 d'options).

Exemples de valeurs :

MSS: 1460 (souvent observé sur Ethernet)

MSS: 1380 (souvent avec VPN ou tunnel)

Rôle :

Indique la **taille maximale de données TCP** (payload) qu'un hôte peut recevoir dans un segment.

Permet d'**éviter la fragmentation IP** : chaque côté s'adapte à la plus petite MSS.

2. Option SACK (Selective Acknowledgment)

Nom du filtre :

tcp.options.sack ou **tcp.options.sack_perm**

Segments concernés :

SACK Permitted → présent dans les **SYN** (ouverture).

SACK → présent dans les **ACK** pendant le transfert de données.

Longueur d'en-tête :

Environ **32 à 40 octets** (selon le nombre de blocs SACK).

Exemples :

SACK Permitted

SACK: Left Edge=1001, Right Edge=2001

Rôle :

Permet de **retransmettre uniquement les segments perdus**, pas tout le bloc.

Améliore l'efficacité du TCP sur les réseaux avec pertes.

3. Option Timestamp (TSval, TSecr)

Nom du filtre :

tcp.options.timestamp.tsval

tcp.options.timestamp.tsecr

Segments concernés :

Présente **dans tous les segments** si activée dès le SYN.

Longueur d'en-tête :

+10 octets (souvent total TCP header = 32 ou 40 octets).

Exemples :

TSval=123456789, TSecr=987654321

✳ Rôle :

Sert à mesurer le **Round Trip Time (RTT)** de manière précise.

Aide à **éviter les vieux segments** (Protection PAWS).

4. Option Window Scale

Nom du filtre :

tcp.options.wscale.shift

Segments concernés :

→ Présente **dans le SYN** (phase d'ouverture).

Longueur d'en-tête :

+3 octets (souvent 32 octets au total).

Exemples :

Window scale: 7 (multiply by 128)

Window scale: 8 (multiply by 256)

✳ Rôle :

Permet d'**augmenter la taille maximale de la fenêtre TCP** au-delà de 65 535 octets.

Utilisé dans les connexions à **haut débit / haute latence** (pour améliorer le débit).

5. Option No Operation (NOP)

Nom du filtre :

tcp.options.nop

Segments concernés :

→ Présente **dans beaucoup de segments**, souvent dans le SYN.

Longueur d'en-tête :

+1 octet (utilisé comme séparateur).

Exemples :

No-Operation (NOP) apparaît souvent entre deux options (ex : entre Timestamp et Window Scale).

✳ Rôle :

Sert uniquement à **aligner** les options TCP sur un multiple de 4 octets (obligation d'alignement).

Ne transporte **aucune donnée**.



Résumé global (tableau)

Option	Présente dans	Longueur TCP (approx.)	Exemple	Rôle principal
MSS	SYN	32 B	MSS=1460	Taille max des données TCP
SACK	SYN, ACK	32-40 B	SACK=1001-2001	Retransmettre seulement les paquets perdus
Timestamp	Tous (si activé)	+10 B	TSval=..., TSecr=...	Mesure RTT + protection PAWS
Window Scale	SYN	32 B	WS=7 (x128)	Augmenter la fenêtre max
No Operation	SYN, souvent	+1 B	NOP	Alignement des options

-- Video D'explication Pas encore fait : j'ai la flemme --

5. Découverte d'UDP

Essayez de trouver des segments UDP dans vos traces. Si vous n'en avez pas, capturez la trace de la navigation vers une page web qui ne se trouverait pas dans vos caches (ex : figaro.fr).

Observez alors l'en-tête UDP des segments capturés. Pour chaque champ ou flag :

Quel est son nom ?

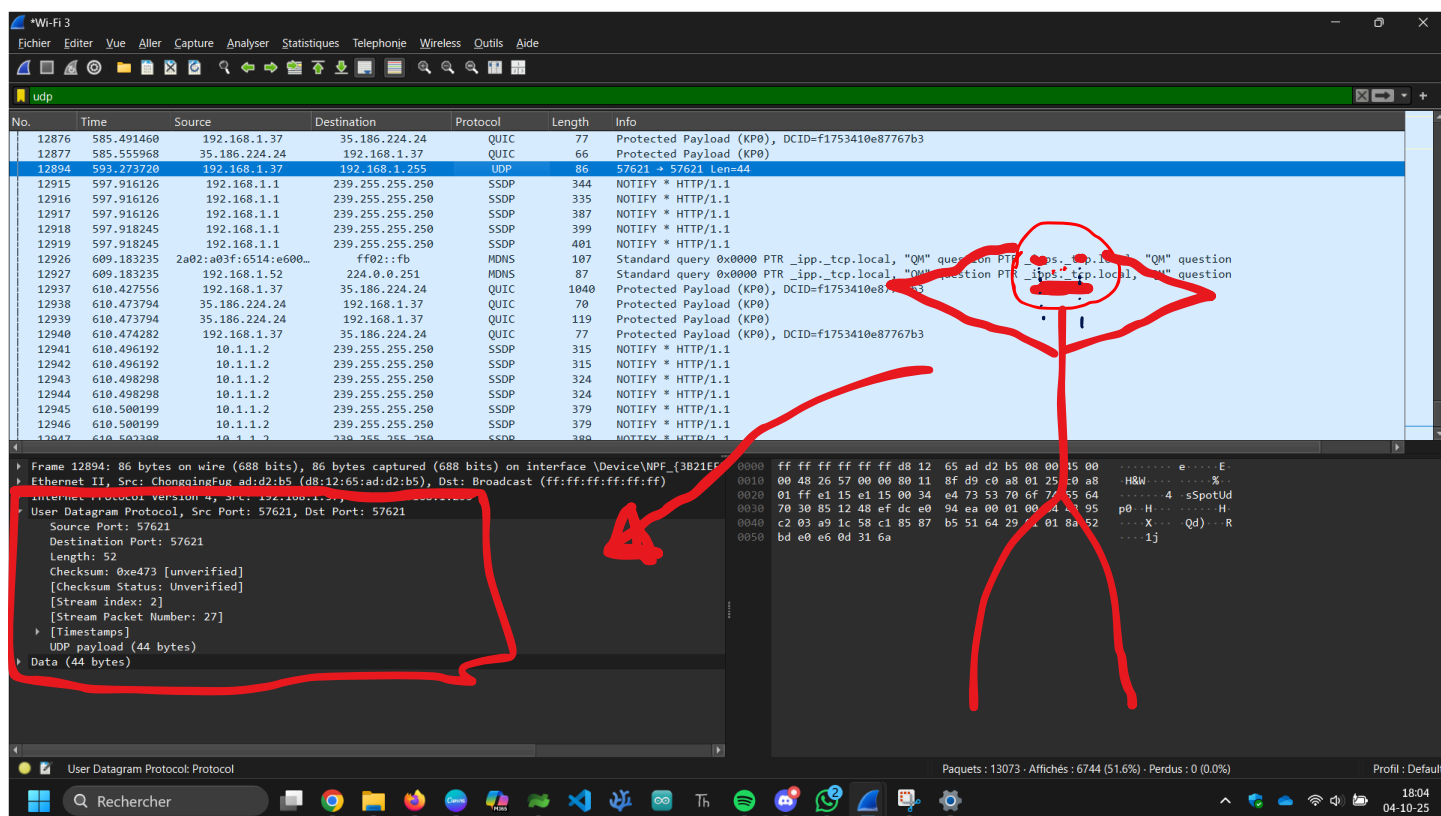
Sur combien de bits est-il encodé ?

Donnez quelques exemples de valeur pour ce champ

Faites une hypothèse sur son usage

Validez/invalides cette hypothèse par une courte recherche Web

Reformulez en deux lignes la signification et l'usage de ce champ



Détail de chaque champ

1. Source Port

Nom : Port source

Taille : 16 bits

Exemples : 12345 (client), 53 (serveur DNS)

Rôle : Permet au destinataire de savoir **quelle application** a envoyé le paquet.

Résumé : C'est le **numéro de port d'envoi**, souvent choisi aléatoirement par le client.

2. Destination Port

Nom : Port de destination

Taille : 16 bits

Exemples : 53 (DNS), 161 (SNMP), 80 (HTTP via UDP/QUIC)

Rôle : Indique **quelle application** ou **service** doit recevoir le paquet.

Résumé : Sert à **diriger le paquet** vers le bon service UDP sur la machine distante.

3. Length

Nom : Longueur

Taille : 16 bits

Exemples : 28, 64, 512

Rôle : Donne la taille **totale du datagramme UDP** (en-tête + données).

Résumé : Permet au récepteur de savoir **combien d'octets** lire.

4. Checksum

Nom : Somme de contrôle

Taille : 16 bits

Exemples : 0x8f3a, 0xd4b1

Rôle : Vérifie que le segment UDP n'a pas été **altéré pendant la transmission**.

Résumé : Sert à **détecter les erreurs** dans l'en-tête et les données.
(Obligatoire en IPv6, optionnelle en IPv4.)



Résumé global (en 2 lignes)

L'en-tête UDP contient 4 champs fixes (source port, destination port, longueur, checksum) codés sur 16 bits chacun.

Ils servent à identifier les applications concernées, la taille du message et à vérifier son intégrité.

Faites ensuite une comparaison entre l'en-tête TCP et l'en-tête UDP : Qu'est ce qui justifie ces différences ?



Comparaison entre l'en-tête TCP et l'en-tête UDP

Caractéristique	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)	Différence / Justification
Taille de l'en-tête	20 octets minimum (jusqu'à 60)	8 octets fixes	TCP contient plus d'informations car il gère la fiabilité et le contrôle du flux.
Connexion	Orienté connexion (établit une session avec 3-way handshake)	Sans connexion	UDP est plus rapide, mais ne garantit pas la livraison.
Fiabilité	Gère les accusés de réception, retransmissions, et ordre des paquets	Aucune garantie de réception ou d'ordre	TCP est fiable, UDP est "best effort".
Contrôle de flux et congestion	Oui (fenêtre, séquençement, etc.)	Non	TCP adapte le débit selon la capacité du réseau.
Usage typique	Transferts de fichiers, web (HTTP, HTTPS), mails, etc.	Vidéo/audio en streaming, DNS, jeux en ligne	UDP est choisi quand la rapidité est plus importante que la fiabilité.

💡 Pourquoi ces différences ?

➡ **TCP** est conçu pour **assurer une communication fiable**, donc il a besoin :

- de numéros de séquence,
- d'accusés de réception,
- de mécanismes de contrôle de flux et de congestion,
- d'une connexion persistante (établie avant d'envoyer des données).

➡ **UDP**, au contraire, est pensé pour **minimiser la latence** :

- il envoie directement les données sans vérification,
- il n'attend aucune réponse,
- il est plus léger et plus rapide,

- parfait pour les flux temps réel (voix, vidéo, jeux...).
-

Vous pouvez également remarquer l'existence du menu "Follow UDP stream". Sur quoi se base Wireshark pour ce filtre ?

Lorsque tu fais **Follow UDP Stream**, Wireshark **reconstitue la conversation** entre deux hôtes.

Ce filtre se base sur les 4 éléments suivants :

- **Adresse IP source**
- **Adresse IP destination**
- **Port source UDP**
- **Port destination UDP**

Autrement dit, Wireshark filtre tous les paquets **ayant le même couple IP/port dans les deux sens**, pour afficher uniquement **ce dialogue spécifique**.

https://www.canva.com/design/DAG0bZmRmCo/TxoWFy2g5yQZbaa-c8nbYw/view?utm_content=DAG0bZmRmCo&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=h873721554c